

Prueba Técnica – Full Stack Advanced (Weather Real-Time Dashboard)

Introducción

Hola, ¡felicitaciones por avanzar en el proceso!

Esta prueba busca evaluar tu capacidad para construir una solución full stack real, integrando:

- Consumo de APIs externas
- Persistencia de datos
- Procesamiento backend
- Tiempo real (WebSockets)
- Estrategias de cache
- Experiencia de usuario (incluyendo modo offline)

Puedes apoyarte en herramientas de IA como Cursor, Antygravity u otras. Lo importante es cómo diseñas y construyes la solución.

Objetivo

Construir un dashboard que permita comparar el clima de dos ciudades utilizando datos de APIS como: [OpenWeather](#), con actualización en tiempo real, persistencia en base de datos y capacidades de exportación.

Descripción funcional

La aplicación debe permitir:

- Seleccionar dos ciudades
- Consultar y almacenar datos climáticos
- Visualizar información en un dashboard
- Comparar datos en gráficos
- Actualizar datos en tiempo real
- Trabajar en modo offline
- Exportar e imprimir información

Flujo del sistema

Frontend → Backend API → Redis Cache → PostgreSQL → API externa (OpenWeather)

↓

WebSocket (tiempo real)

Backend (Django + DRF)

♦ Tecnologías requeridas

- Django
- Django REST Framework
- PostgreSQL
- Redis (cache obligatorio)
- WebSockets (Django Channels)
- Celery (opcional, altamente valorado)

♦ **Modelo principal**

WeatherData

- id
- city
- temperature
- humidity
- wind_speed
- timestamp

♦ **Funcionalidades requeridas**

1. **Integración con API externa**

- Consumir datos desde OpenWeather (se puede usar cualquier otra API meteorológica)
- Manejar errores y caídas de la API

2. **Persistencia**

- Guardar histórico de datos climáticos
- Permitir consultas eficientes

3. **Cache (Redis)**

- Cachear datos recientes por ciudad
- Definir estrategia de expiración (TTL)

4. **API REST**

Endpoints mínimos:

- GET /weather/ (paginado)
- GET /weather?city=...
- Endpoint optimizado /dashboard-data/

5. **Paginación (IMPORTANTE)**

- Backend debe implementar paginación (limit/offset o page/size)
- Debe ser consumida correctamente por el frontend

6. **Exportación CSV**

- Endpoint para descargar datos en formato CSV

7. **Tiempo real**

- Emitir eventos cuando se inserten nuevos datos
- Canal WebSocket para el frontend

8. **Manejo de errores**

- Fallback a datos en DB si falla API externa
- Logs claros

9. **Seguridad (sin autenticación)**

- Validación de inputs
- Prevención básica de:
 - SQL Injection

- XSS
- CSRF (si aplica)

Frontend (React + Vite + Ant Design)

♦ Requerimientos

- React + Vite
- Ant Design (Biblioteca de componentes UI a su elección)
- Manejo de estado (Redux, Zustand, useState o similares)

♦ Funcionalidades

1. Dashboard

- Comparación de dos ciudades
- Cards con:
 - Temperatura
 - Humedad
 - Viento

2. Tabla de datos

- Mostrar datos de **WeatherData**
- Paginación:
 - Backend + Frontend
- Filtros por ciudad

3. Gráficos

- Series de tiempo
- Comparación entre ciudades

4. Tiempo real

- Conexión WebSocket
- Actualización automática de UI

5. Exportación

- Descargar datos en CSV de la tabla **WeatherData**

6. Impresión

- Opción para imprimir el dashboard

7. Modo offline

Cuando no haya conexión:

- Mostrar último estado disponible
- Usar almacenamiento local (localStorage o IndexedDB)
- Mostrar indicador visual de “offline”
- Re-sincronizar al recuperar conexión

8. UX/UI

- Loading states
- Manejo de errores
- Diseño responsive

Infraestructura

♦ Requerido

- Docker
- `docker-compose`
- Servicios:
 - Backend
 - Frontend
 - Redis
 - PostgreSQL

♦ Despliegue

- Deploy en Heroku o herramienta de despliegue a su elección

Bonus (muy valorado)

- Pruebas unitarias
- Manejo de reconexión WebSocket
- Diagrama de arquitectura

Entregables

1. Repositorio público en GitHub que incluya:
 - Código fuente
 - README con:
 - Arquitectura
 - Decisiones técnicas
 - Instrucciones de ejecución
2. Aplicación desplegada (URL pública)
3. Un video explicando el flujo de desarrollo(código) y explicación de UI(aplicativo). (El video puede ser publicado en Google Drive y se debe compartir la url)

Criterios de evaluación

Arquitectura

- Organización del código
- Separación de responsabilidades

Performance

- Uso de cache
- Eficiencia en consultas

Tiempo real

- Implementación WebSocket

Resiliencia

- Manejo offline
- Manejo de errores

Frontend

- UX/UI
- Manejo de estado
- Componentes Dinámicos

DevOps

- Docker
- Despliegue

Tiempo estimado

Fecha límite de entrega:

Consideraciones finales

- No se evaluará autenticación
- Sí se evaluarán buenas prácticas de seguridad
- Se valora claridad en decisiones técnicas

¡Muchos éxitos!

Queremos ver cómo diseñas una solución real, escalable y bien pensada.